

MODULE 03

CNN 합성곱 신경망: 6×6 픽셀부터 손글씨 판별 까지

작은 돋보기(커널)로 선을 찾고, 꺾꺾 눌러 압축한 뒤,
숫자 1과 2를 판별하는 AI의 9단계 계산 과정

🕒 약 50분

📁 엑셀 + Colab

📖 선수 지식: 사칙연산이면 OK

준비물: '엑셀' 실습 파일과 코랩 노트북을 함께 열어 두세요. · 선수 지식: 파이썬을 처음 봐도 괜찮아요! 사칙연산 정도면 충분해요. 😊

6×6 픽셀 그리기

이진 모드 (0/255)

		255	255		
		255	255		
		255	255		
		255	255		
		255	255		
		255	255		

붓 밝기 (값): 255 ☒ 이진 모드(흑/백)

숫자 1 프리셋 숫자 2 프리셋

대각선 (/) 십자가 (+)

✓ 캔버스 전체 지우기

1 입력 이미지 정규화 (Normalization) : $(0 \sim 255 \rightarrow 0.0 \sim 1.0)$

정규화된 6×6 행렬 (입력 $\{X_i\}$)

0	0	1.0	1.0	0	0
0	0	1.0	1.0	0	0
0	0	1.0	1.0	0	0
0	0	1.0	1.0	0	0
0	0	1.0	1.0	0	0
0	0	1.0	1.0	0	0

정규화 공식:
$$\{ \text{정규화 픽셀 값} \} = \frac{\{ \text{원본 픽셀 값} \}}{255}$$

예: 검은 바탕 $\{0 \rightarrow 0.0\}$, 흰 글씨 $\{255 \rightarrow 1.0\}$, 회색 $\{128 \rightarrow 0.50\}$

255로 나누어 계산을 가깝게 만들

2 특징 추출 필터 4종 (3×3 커널)

선과 대각선 특징을 찾아내는 돋보기

필터 1: 세로선 (|)

0	1	0
0	1	0
0	1	0

세로 획 강조

필터 2: 가로선 (-)

0	0	0
1	1	1
0	0	0

가로 획 강조

필터 3: 슬래시 (/)

0	0	1
0	1	0
1	0	0

오른쪽 위 대각선

필터 4: 백슬래시 (\)

1	0	0
0	1	0
0	0	1

왼쪽 위 대각선

▲ CNN 계산 시뮬레이터 — 그린 숫자가 9단계 파이프라인을 거쳐 판별됩니다

START · 이 모듈의 지도

9단계 파이프라인을 따라가면 손글씨 판별 AI가 완성됩니다

이 모듈에서는 컴퓨터 비전을 한 단계 끌어올립니다. 지난 모듈에서 사람이 직접 알려준 '이웃 뺄셈' 대신, AI가 스스로 특징을 찾아내는 구조 — **CNN(합성곱 신경망)**을 배웁니다. 6×6 픽셀에 숫자를 그리면 9단계 계산을 거쳐 '1인지 2인지' 판별하는 전 과정을 실시간으로 볼 수 있습니다.

9단계 파이프라인 미리 보기

- 1 **입력 이미지**
6×6 픽셀에 손으로 숫자를 그립니다.
- 2 **정규화**
0~255 값을 0.0~1.0 소수로 바꿔 계산을 가볍게 합니다.
- 3 **합성곱**
4종 커널(돋보기)로 선·모양을 찾아 4×4 특성맵 4개를 만듭니다.
- 4 **맥스풀링**
가장 강한 특징만 남기고 2×2로 압축합니다.
- 5 **평탄화**
2D 맵을 1D 벡터(8개 숫자)로 펼칩니다.
- 6 **완전연결층**
가중치 곱 + 편향으로 최종 점수를 계산합니다.
- 7 **판별**
점수 $\geq 0 \rightarrow$ 숫자 1, 점수 $< 0 \rightarrow$ 숫자 2

이 모듈에서 배우는 것 (3줄 요약)

- ① 컴퓨터가 6×6 픽셀 그림을 **숫자 행렬**로 읽는 방법을 알아봅니다.
- ② 작은 필터(**커널**: 모양 찾기 돋보기 )로 선·모양을 찾아내는 **합성곱**을 직접 계산해 봅니다.
- ③ 찾은 특징을 꼭꼭 눌러 **압축(풀링·평탄화)**한 뒤 숫자 1과 2를 판별하는 전체 과정을 완성합니다.

준비 완료 체크

- 엑셀 실습 파일(CNN_계산과정_숫자판별_정리본.xlsx)과 Colab 노트북을 함께 열어 두었다.
- 시뮬레이터의 [실시간 CNN 계산기] 탭을 열어 두었다.

도입 · 질문

이미지를 1줄로 펼치면 무엇이 사라질까?

기존 신경망은 사진을 **1줄로 길게 펼쳐서** 입력받았습니다.
그때 함께 뭉개져 버린 것은?

일반 인공신경망(MLP)은 2차원 이미지를 1줄로 펼쳐 입력받았기 때문에, **픽셀들 사이의 상하좌우 공간 관계(선, 모서리, 모양)**가 파괴되는 심각한 단점이 있었습니다. 눈·코·입의 위치 관계가 흩어져 버리면 얼굴을 알아보기가 훨씬 어려워지죠.

CNN의 아이디어 — 사람 눈의 방식을 본뜨다

CNN(Convolutional Neural Network)은 사람이 눈으로 사물을 볼 때 **윤곽선이나 특징을 먼저 파악**하는 원리를 본떴습니다. 작은 필터(커널)를 이미지 전체에 슬라이딩(합성곱)시키며, 위치와 상관없이 사물의 고유한 특징을 찾아내는 혁신적인 구조입니다.

비유 — 돋보기로 그림 훑어보기

3×3 크기의 작은 돋보기를 들고 6×6 그림의 왼쪽 위부터 오른쪽 아래까지 1칸씩 이동하며, 돋보기 속의 모양(세로선, 가로선 등)과 일치하는 부분이 어디에 있는지 찾아 점수로 기록하는 과정 — 그것이 바로 **합성곱(Convolution)**입니다.

교사용 팁 — 수업 운영

학생들에게 숫자 '1'을 위아래로 뒤집어 그려 보게 하세요. 사람은 여전히 1로 읽지만 위치가 바뀌었다는 점에 서, CNN이 '위치와 상관없이 특징을 찾는' 이유를 실감할 수 있습니다.

실시간 CNN 계산기 (6x6 드로잉)

 CNN 9단계 원리 및 공식 해설 학생 탐구 도전 과제 (4단계)

6×6 픽셀에 숫자를 그리고 CNN의 9단계 계산 과정을 실시간으로 관찰하세요!

왼쪽 캔버스에 마우스를 클릭하거나 드래그하여 숫자를 그리면, 정규화 → 4중 커널 합성곱 → 벡스플링 → 평탄화(벡터화) → 가중치(sws)와 편향(bs) 내적을 거쳐 컴퓨터가 '1'인지 '2'인지 판별하는 전체 수학적 과정이 실시간으로 계산됩니다.

이 모듈에서 배우는 것 (3줄 요약)

1. 컴퓨터가 6×6 픽셀 그림을 숫자 행렬로 읽는 방법을 알아봐요.
2. 작은 필터(커널: 모양 찾기 돋보기 )로 선·모양을 찾아내는 **합성곱**을 직접 계산해 봐요.
3. 찾은 특징을 꼭꼭 눌러 압축(풀링·평탄화)한 뒤, 숫자 1과 2를 판별하는 AI의 전체 과정을 완성해요!

▲ 모듈 3 도입 — 6×6 픽셀을 읽고 CNN이 숫자를 판별하는 전 과정

개념 ①

커널 — 모양 찾기 돋보기 4종

커널(Kernel)은 3×3 크기의 작은 숫자 행렬로, '어떤 모양을 찾을 것인가'를 정해 놓은 돋보기입니다. 이 모듈에서는 선 4종을 찾는 커널을 사용합니다.

필터	3×3 커널	찾는 모양
필터 1	$\begin{matrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{matrix}$	세로선 () — 숫자 1의 고유 특징
필터 2	$\begin{matrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{matrix}$	가로선 (—) — 숫자 2의 고유 특징
필터 3	$\begin{matrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{matrix}$	슬래시 (/) — 숫자 2의 대각선
필터 4	$\begin{matrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{matrix}$	백슬래시 (\) — 숫자 2의 대각선

커널은 어떻게 '찾아내는가'?

커널을 이미지의 한 위치에 겹쳐 놓고 대응하는 칸끼리 곱한 뒤 모두 더합니다. 모양이 일치할수록 큰 점수 — 이미지 전체를 훑으며 '세로선 지도'를 그립니다.

0

0

1.0

1.0

0

0

2 특징 추출 필터 4종 (3×3 커널)

선과 대각선 특징을 찾아내는 돋보기

필터 1: 세로선 (|)

$$\begin{matrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{matrix}$$

세로 획 강조

필터 2: 가로선 (—)

$$\begin{matrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{matrix}$$

가로 획 강조

필터 3: 슬래시 (/)

$$\begin{matrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{matrix}$$

오른쪽 위 대각선

필터 4: 백슬래시 (\)

$$\begin{matrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{matrix}$$

왼쪽 위 대각선

▲ 4종 커널 — 세로선·가로선·대각선 2종을 찾는 돋보기

AI Life — 인공지능과 일상

외부 강의용 교재

개념 ②

합성곱 — 원소별 곱셈 후 합산

커널을 이미지 위에서 1칸씩 이동(슬라이딩)시키며 각 위치의 점수를 계산하는 연산이 **합성곱 (Convolution)**입니다.

$$O(i, j) = \sum \sum I(i+m, j+n) \times K(m, n)$$

입력 이미지 I의 (i,j) 위치에서 3×3 커널 K와의 합성곱 — 대응 원소끼리 곱해 모두 더한 값



크기는 어떻게 될까?

입력 6×6, 커널 3×3, 보폭(Stride) 1, 패딩 0일 때 출력 특성맵의 크기는 $(6 - 3) + 1 = 4 \times 4$ 가 됩니다. 커널이 이미지 안에서 움직일 수 있는 자리 수가 곧 출력 크기입니다.



수용장(Receptive Field) 실시간 계산기

시뮬레이터의 4×4 특성맵 셀 위에 마우스를 올리면, 그 셀을 만든 6×6 이미지의 3×3 영역과 원소별 곱셈·합산 과정이 그대로 표시됩니다. 수식 $O(i,j)$ 가 눈앞에서 살아 움직이는 순간입니다 — 꼭 체험해 보세요!



판별 원리 미리 보기: 숫자 1을 그리면 세로선 필터(필터 1)의 반응값이 높아집니다. 이 반응값에 양수 가중치(+3)를 곱하면 높은 양의 점수가 나와 '숫자 1'로 판별됩니다.

개념 ③

맥스풀링 — 가장 강한 것만 남기고 꺾꺾 압축

맥스풀링(Max Pooling)은 4×4 특성맵을 2×2 구역 4개로 나누고, 각 구역에서 **가장 큰 값 하나만** 남기는 연산입니다.

$$P(r, c) = \max \{ O(2r+m, 2c+n) \}$$

2×2 풀링, 보폭 2 — 4×4 특성맵 $\rightarrow 2 \times 2$ 풀링맵



풀링이 주는 두 가지 선물

① 계산량 75% 감소

16개 숫자 $\rightarrow$ 4개 숫자. AI의 머릿속이 가벼워집니다.

② 위치 불변성

글씨가 살짝 빠뚫어지거나 이동해도 가장 강한 특징은 살아남아 안정적으로 감지됩니다.



비유 — 반찬 압축

김치통을 꺾꺾 눌러 부피를 줄여도 김치의 맛(핵심)은 그대로죠. 풀링도 마찬가지입니다. '가장 강한 맛(특징)'만 남기고 부피를 줄이는 거예요.

개념 ④

평탄화 → 완전연결층 → 최종 판별

압축된 특징들을 모아 최종 판단을 내립니다. 여기서부터는 모듈 1의 단일 뉴런과 같은 계산이 등장합니다.

1 평탄화(Flattening)

2D 풀링맵 4개를 1D 벡터로 펼칩니다. 엑셀 실습 방식대로 각 풀링맵에서 상단 2개씩, 총 **8개 특징**을 추출합니다.

2 가중치 내적

8차원 특징 벡터 X에 학습된 가중치 W를 곱하고 편향 b를 더합니다.

3 최종 판별

점수가 0 이상이면 숫자 1, 0 미만이면 숫자 2로 판별합니다.

$$\text{Score} = \sum (X_i \times W_i) + b \quad \rightarrow \quad \text{Score} \geq 0 \text{ 이면 } 1, \text{ 아니면 } 2$$

예: 세로선 특징 $\times (+3)$ + 가로선 특징 $\times (-3)$ + ... + 편향(+5.0)

💡 왜 가중치 부호가 서로 다를까?

세로선 필터(1~2번)는 숫자 1의 고유 특징이므로 **양수 가중치(+3)**를 곱해 점수를 올립니다.

가로선·대각선 필터(3~6번)는 숫자 2의 고유 특징이므로 **음수 가중치(-3)**를 곱해, 이 모양이 있으면 점수를 크게 깎아 '숫자 2'로 판별되게 합니다!

즉, AI는 '**1의 증거는 더하고, 2의 증거는 뺀다**'는 전략으로 판단합니다.

체험 ① · 시뮬레이터

6×6 픽셀에 숫자를 그려라

왼쪽 캔버스에 마우스를 클릭하거나 드래그하여 숫자를 그리면, 그 아래 1~6단계 파이프라인에 실시간으로 반영됩니다.

1 그리기

1 숫자 1 · 2 숫자 2 프리셋을 누르거나 직접 드래그로 그립니다.  대각선,  십자가 프리셋도 있습니다.

2 정규화 확인

그린 그림이 0~255 → 0.0~1.0 소수로 변환된 정규화된 6×6 행렬로 표시됩니다.

3 예측 결과

상단의 최종 예측 결과(숫자 1/2, 점수, 확률)가 실시간으로 바뀝니다.



정규화 공식

$$\text{정규화 픽셀 값} = \text{원본 픽셀 값} \div 255$$

예: 검은 바탕 0 → 0.0, 흰 글씨 255 → 1.0, 회색 128 → 0.50



체험 ② · 시뮬레이터

4종 커널의 합성곱을 실시간으로

숫자를 그리면 4개의 커널이 동시에 작동하며 4×4 특성맵 4개를 만들어냅니다. 각 특성맵의 최대값이 함께 표시됩니다.

관찰 포인트

숫자 1을 그리면 → 특성맵 1(세로)의 최대값이 가장 큼니다. (예: 3.0)

숫자 2를 그리면 → 특성맵 2(가로)·3·4(대각선)의 값들이 커집니다.

각 필터가 '자기 모양'을 찾아내는 장면을 직접 눈으로 확인하는 것이 이 단계의 핵심입니다.

수용장 계산기로 수식 검증하기

특성맵의 셀 위에 마우스를 올리면 3×3 영역 $\times$ 커널의 원소별 곱셈·합산 과정이 표시됩니다. 개념 ②에서 본 공식 $O(i,j) = \sum \sum I \times K$ 가 그대로 재현되는 것을 확인해 보세요.

교사용 팁 — 수업 운영

학생들에게 '특성맵 1의 최대값이 3.0이 나오는 3×3 영역을 찾아보라'고 하세요. 수용장 계산기로 역추적하는 과정에서 합성곱의 의미가 완전히 이해됩니다.

3 합성곱 연산 (Convolution) $\rightarrow$ 4×4 특성맵 4개

보폭(Stride) = 1, 패딩 없음

특성맵 1 (세로)

0.0	3.0	3.0	0.0
0.0	3.0	3.0	0.0
0.0	3.0	3.0	0.0
0.0	3.0	3.0	0.0

최대값: 3.0

특성맵 2 (가로)

1.0	2.0	2.0	1.0
1.0	2.0	2.0	1.0
1.0	2.0	2.0	1.0
1.0	2.0	2.0	1.0

최대값: 2.0

특성맵 3 (대각선 /)

1.0	2.0	2.0	1.0
1.0	2.0	2.0	1.0
1.0	2.0	2.0	1.0
1.0	2.0	2.0	1.0

최대값: 2.0

특성맵 4 (역대각선 \)

1.0	2.0	2.0	1.0
1.0	2.0	2.0	1.0
1.0	2.0	2.0	1.0
1.0	2.0	2.0	1.0

최대값: 2.0

수용장(Receptive Field) 실시간 계산기:

위 4×4 특성맵의 셀 위에 마우스를 올리면, 해당 위치를 만든 6×6 이미지의 3×3 영역과 원소별 곱셈 합산 과정이 여기에 표시됩니다!

▲ 합성곱 실시간 계산 — 4종 커널이 만드는 4개의 특성맵

체험 ③ · 시뮬레이터

풀링 → 평탄화 → 점수 판별의 완결

파이프라인의 뒷부분입니다. 4×4 특성맵이 2×2 풀링맵으로 압축되고, 8개 특징 벡터가 되어 최종 점수로 판별됩니다.

1 맥스풀링 확인

4개의 2×2 풀링맵에서 가장 강한 특징만 살아남았는지 봅니다.

2 평탄화

4개 풀링맵에서 상단 2개씩 뽑아 8차원 특성 벡터 $X = [x_1 \cdots x_8]$ 이 됩니다.

3 가중치 곱 표

특징 번호 · 출처 필터 · 입력 벡터 · 가중치 · 가중치 곱이 한눈에 보이는 표를 읽습니다.

4 최종 판별

가중치 곱 총합 + 편향(+5.0) = 최종 점수. 0 이상이면 숫자 1!



점수 합산표 읽기

표의 마지막 행: 가중치 곱 총합($\sum X \times W$) + 편향(+5.0) = 최종 점수. 0 이상이면 숫자 1! 모듈 1의 단일 뉴런($\sum X \times W + b$)과 같은 구조입니다.

5 평탄화(Flattening): 2D 풀링맵 → 1D 특성 벡터 변환

엑셀 실습 방식: 4개 풀링맵에서 상단 2개씩 총 8개 특징 추출

최종 8차원 특성 벡터 $(X = [x_1, x_2, \dots, x_8])$

x1(세로선1): 3.0

x2(세로선2): 3.0

x3(가로선1): 2.0

x4(가로선2): 2.0

x5(대각선(/)1): 2.0

x6(대각선(/)2): 2.0

x7(역대각선(\)1): 2.0

x8(역대각선(\)2): 2.0

6 완전연결층(Dense Layer) 연산 : $\sum (X_i \times W_i) + b$

가중치 곱과 편향 덧셈을 통한 최종 판별 점수 산출

특징 번호	출처 필터	입력 벡터 $(\backslash(X_i))$	가중치 $(\backslash(W_i))$	가중치 곱 $(\backslash(X_i \times W_i))$	설명
-------	-------	---------------------------	-------------------------	--------------------------------------	----

▲ 완전연결층 연산 — 8개 특징의 가중치 곱과 편향으로 최종 판별

코드 ① · 파이썬

합성곱과 풀링을 코드로 정의하기

시뮬레이터의 계산 과정을 순수 NumPy로 그대로 옮긴 코드입니다.

```
# 1. 2D 합성곱 (Convolution: 6x6 -> 4x4)
def conv2d(image, kernel):
    h, w = image.shape
    kh, kw = kernel.shape
    out = np.zeros((h - kh + 1, w - kw + 1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = np.sum(image[i:i+kh, j:j+kw] * kernel)
    return out

# 2. 2D 맥스풀링 (Max Pooling 2x2, Stride=2: 4x4 -> 2x2)
def max_pool2d(feature_map, pool_size=2):
    h, w = feature_map.shape
    out = np.zeros((h // pool_size, w // pool_size))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = np.max(
                feature_map[i*pool_size:(i+1)*pool_size,
                           j*pool_size:(j+1)*pool_size])
    return out
```

 (실습)_CNN_계산과정.ipynb — 합성곱·풀링 셀

🔍 코드 읽기 포인트

- ① **image[i:i+kh, j:j+kw] * kernel**: 3×3 영역과 커널의 원소별 곱셈 — 개념 ②의 공식 $O(i,j) = \sum I \times K$ 를 그대로 코드로 옮긴 것입니다.
- ② **np.max(...)**: 2×2 구역에서 가장 큰 값 하나만 남기기 — 풀링의 정의 그대로입니다.
- ③ **for i, for j**: 커널이 이미지를 훑는 '슬라이딩'을 이중 반복문으로 표현했습니다.

코드 ② · 파이썬

판별 파이프라인 완성 — predict_digit

앞의 함수들을 조립하면 손글씨 판별기 전체가 완성됩니다.

```
# 3. 4중 특징 필터 정의 (세로, 가로, 대각선 2중)
kernels = [
    np.array([[0,1,0],[0,1,0],[0,1,0]]), # 1) 세로선 필터 (|)
    np.array([[0,0,0],[1,1,1],[0,0,0]]), # 2) 가로선 필터 (-)
    np.array([[1,0,0],[0,1,0],[0,0,1]]), # 3) 대각선 필터 (\)
    np.array([[0,0,1],[0,1,0],[1,0,0]]) # 4) 역대각 필터 (/)
]

# 4. 완전연결층 가중치 및 편향
weights = np.array([3.0, 3.0, -3.0, -3.0, -2.0, -2.0, -1.0, -1.0])
bias = 5.0

# 5. 순전파 및 최종 숫자 판별 함수
def predict_digit(image_6x6):
    norm_img = image_6x6 / 255.0 # 정규화
    flat_vector = []
    for k in kernels:
        fmap = conv2d(norm_img, k) # 합성곱
        pooled = max_pool2d(fmap) # 풀링
        flat_vector.extend([pooled[0,0], pooled[1,1]]) # 평탄화
    flat_vector = np.array(flat_vector)
    score = np.dot(weights, flat_vector) + bias
    label = 1 if score >= 0 else 2
    return label, score
```

 (실습)_CNN_계산과정.ipynb — 판별 셀 (발체)

✅ 여기까지 되면 ✅

- conv2d → max_pool2d → 평탄화 → np.dot + bias 순서가 파이프라인 9단계와 일치함을 설명할 수 있다.
- weights의 +3/-3 부호가 '1의 증거는 더하고 2의 증거는 뺀다'는 전략임을 설명할 수 있다.

심화 탐구

학생 탐구 도전 과제 4단계

시뮬레이터의 [ 학생 탐구 도전 과제] 탭에 있는 4가지 실험입니다. CNN의 한계와 가능성을 직접 부딪혀 보는 시간입니다.

도전 1 — 세로선을 오른쪽으로 옮기면?

[1 숫자 1 프리셋] 후 세로선 2줄을 오른쪽으로 1~2칸 옮겨 그려 보세요. 위치가 바뀌어도 '숫자 1'로 잘 인식될까요? 맥스풀링의 위치 불변성 효과를 관찰하는 실험입니다.

도전 2 — 대각선(/)만 그리면?

대각선만 1줄 그려 보세요. 대각선 필터(필터 3)의 값은 치솟지만, 가중치 테이블에서 대각선 가중치는 음수(-3)입니다. 점수가 어떻게 깎여서 최종 판별이 나오는지 점수 합산표를 분석해 보세요.

도전 3 — '1'도 '2'도 아닌 글자를 그리면?

+ 십자가나 동그라미를 그려 보세요. 이 모델은 1과 2만 구별하도록 훈련됐습니다. 완전히 새로운 모양이 들어왔을 때 AI는 왜 엉뚱한 예측을 낼 수밖에 없을까요? 예측 확률(신뢰도)의 한계에 대해 토론해 봅시다.

도전 4 — 가중치와 편향을 직접 바꾸면?

편향(+5.0)을 -10.0으로 대폭 낮추면 어떻게 될까요? 아주 굵은 1이 아니면 전부 2로 판별하게 될까요? 파이썬 노트북에서 가중치 값을 직접 변경해 보는 실험을 진행해 봅시다.

교사용 팁 — 토론 주제

도전 3은 AI의 가장 중요한 한계를 드러냅니다: **AI는 배운 것만 안다**. 이 한계를 아는 것이 AI를 현명하게 쓰는 첫걸음입니다.

연계 · 세상 속 AI

엑셀에서 파이썬으로, 교실에서 세상으로

 엑셀(Excel) ↔ 파이썬(Colab) 연계표

CNN 단계	엑셀 수식	파이썬 코드	교육적 의미
정규화	=A5/255	img / 255.0	0~255를 0~1 소수로
합성곱	=SUMPRODUCT(3×3영역, 커널)	np.sum(영역 × 커널)	선/형태 패턴 매칭 점수
맥스풀링	=MAX(2×2영역)	np.max(2×2영역)	가장 강한 특징만 압축
가중치 내적	=SUMPRODUCT(벡터, 가중치)+편향	np.dot(벡터, W) + b	종합 점수 산출
최종 분류	=IF(점수≥0, 1, 2)	1 if score ≥ 0 else 2	양수/음수 기준 판별

같은 계산을 **엑셀 수식**과 **파이썬 코드** 두 가지 언어로 표현하면, '계산의 본질은 하나'라는 것을 깨닫게 됩니다. 도구는 달라도 수학은 같습니다.

 세상 속 CNN

 **얼굴 인식 잠금해제:** 눈·코·입의 특징을 커널들이 찾아내 본인인지 판단합니다.

 **자율주행:** 차선·표지판·보행자를 실시간으로 인식합니다.

 **의료 영상 진단:** X-ray·CT 사진에서 병변의 특징을 찾아 의사를 돕습니다.

 **사진 검색:** '고양이 사진 찾아줘'라는 말에 사진 속 특징을 인식해 찾아줍니다.

Q&A

자주 묻는 질문 — 한 걸음 더 깊이

? 커널의 숫자(0과 1)는 누가 정해주나요?

이 모듈에서는 사람이 직접 정해 주었습니다(세로선·가로선 모양). 하지만 진짜 CNN에서는 AI가 **스스로 학습**합니다. 수많은 사진을 보여주며 '이건 고양이, 이건 강아지'를 반복하면, 경사하강법(모듈 1!)이 커널의 숫자들을 조금씩 고쳐 고양이 귀 모양, 강아지 코 모양을 찾는 필터로 만들어냅니다. **손으로 설계한 필터에서 스스로 배우는 필터로** — 그것이 딥러닝의 도약입니다.

? 왜 하필 3×3 인가요? 더 크면 안 되나요?

클 수도 있습니다(5×5 , 7×7). 하지만 커널이 크면 계산량이 늘고, 작은 특징을 놓치기 쉽습니다. 대신 **작은 커널을 여러 층 쌓으면** 넓은 영역도 볼 수 있으면서 세밀함도 유지됩니다. '작은 돋보기를 여러 겹'이 실전 CNN의 설계 철학입니다.

? 풀링에서 최대값이 아닌 평균값을 쓰면요?

그것도 있습니다! **평균 풀링(Average Pooling)**이라고 합니다. 최대값은 '가장 강한 특징'에 집중하고, 평균값은 '전체 분위기'를 봅니다. 무엇을 남길 것인가는 문제에 따라 선택하는 설계 결정입니다.

? 1과 2 말고 0~9全部를 판별하려면?

출력 판별기를 10개로 늘리면 됩니다. 각 숫자마다 '이 숫자의 증거'를 모으는 가중치 세트를 두고, 10개의 점수 중 가장 높은 것을 고르는 방식입니다. 구조는 오늘 배운 것과 정확히 같습니다 — **규모만 커질 뿐**입니다.

정리 · 체크리스트

여기까지 되면 — 모듈 3 완주!

6×6 픽셀에서 시작한 여정이 손글씨 판별 AI로 끝났습니다. 아래를 확인해 보세요.

 핵심 개념 체크

- 커널이 3×3 모양 찾기 돋보기임을 설명할 수 있다.
- 합성곱이 '원소별 곱셈 후 합산'이며 $6 \times 6 \rightarrow 4 \times 4$ 가 됨을 설명할 수 있다.
- 맥스풀링이 가장 강한 특징만 남기는 압축(위치 불변성)임을 설명할 수 있다.
- 평탄화 → 가중치 내적 → 판별의 흐름을 설명할 수 있다.
- 가중치 부호(+3/-3)의 의미('1의 증거는 더하고, 2의 증거는 뺀다')를 설명할 수 있다.

 체험·코드 체크

- 직접 그린 숫자가 실시간으로 판별되는 것을 보았다.
- 수용장 계산기로 합성곱 수식을 검증해 보았다.
- conv2d/max_pool2d/predict_digit 코드의 흐름을 따라갈 수 있다.
- 도전 과제 중 1개 이상을 실험하고 결과를 말할 수 있다.

 한 문장 정리

CNN = 작은 돋보기(커널)로 특징을 찾고(합성곱), 꺾꺾 눌러 압축한 뒤(풀링·평탄화), 가중치 투표로 판단하는 것.

 교사용 팁 — 다음 차시 예고

다음 모듈에서는 이미지에서 글자로 넘어갑니다. 컴퓨터가 한글을 어떻게 숫자로 바꾸는지 — 자연어처리와 AI 에이전트의 세계입니다.